卓胜私募-笔试 耿文博

基础知识

1. C++ Python 分别属于哪种语言

    a. **C++:** C++ 是一种编译型语言，源代码在编译时被转换为机器码，然后执行，并支持多范式语言，支持面向过程、面向对象和泛型编程。属于静态类型语言，需要在编译时指定变量的类型，并进行类型检查。

    b. **Python:** Python 是一种解释型语言，源代码逐行解释并执行，并且也支持面向对象、面向过程、函数式编程的多范式语言，动态类型语言，变量的类型在运行时被确定，无需在编写代码时指定。

2. 简要说明 C++ 编译器的工作原理

    预处理 → 编译 → 汇编 → 链接

    预处理阶段是在源代码被实际编译之前进行的。主要任务包括处理以 `#` 开头的预处理指令，如 `#include`、`#define` 等，处理后的代码被保存为一个扩展名为 `.i` 的中间文件 → 编译阶段将预处理后的代码转换成汇编代码 → 汇编阶段将汇编代码翻译成机器码或目标代码,生成的目标代码通常保存为扩展名为 `.o` 的文件 → 链接阶段将多个目标文件以及可能的库文件组合成一个可执行文件或库。最终生成的可执行文件通常具有扩展名为 `.exe` 的执行文件。

3. 简要说明动态链接库，静态链接库的区别

    静态链接库：静态库在编译时被链接到可执行文件中。 静态库以 `.lib`（Windows）或 `.a`（Unix/Linux）等格式存在。在编译时被复制到可执行文件中，使得可执行文件变得较大。

    动态链接库：动态库在运行时被加载到内存中，而不是在编译时链接到可执行文件中。主要以 `.dll`（Windows），但代码和数据不被复制到可执行文件中，而是在程序运行时由操作系统动态加载到内存。

4. 简要说明 vector，list 的区别

    - `vector`：使用动态数组实现，元素在内存中是连续存储的。
    - `vector`：提供随机访问能力，通过索引可以直接访问元素。
    - `vector`：内存分配连续，对于大量的元素，可能导致频繁的重新分配和拷贝，但在访问元素时效率高。
    - `vector`：插入或删除元素可能导致其他元素的移动，特别是在中间或开头位置。
    - `list`：使用双向链表实现，元素在内存中不一定是连续存储的。
    - `list`：不支持随机访问，必须通过迭代器进行顺序访问。
    - `list`：插入和删除元素更为高效，因为只需要调整相邻元素的指针，不需要移动实际的元素。

- `list`：链表结构在插入和删除操作上更为高效，可能占用更多的内存，并且在内存中的元素分布比较分散。

1. vector.resize(n) 和 vector.reverse(n) 的区别
   - `resize` 修改 `vector` 的大小，可能会改变其中元素的值，而且会在需要的情况下添加或删除元素。
   - `reserve` 仅仅是预留存储空间，并不改变 `vector` 的大小，不会影响其中的元素值。

5. 实现模块，输入 vector<int> 返回不包括偶数的 vector

```
vector<int> filterOddNumbers(const vector<int>& inputVector) {
    vector<int> resultVector;

    for (int num : inputVector) {
        if (num & 1)
            resultVector.emplace_back(num);
    }

    return resultVector;
}
```

   1. 代码不足在哪儿，进一步改进

      可以使用函数模板使函数适用于不同类型的元素。

6. map，unordered_map 的区别

   `map` 底层使用红黑树（Red-Black Tree）实现。由于是一种自平衡二叉搜索树，所以 map 里保持元素有序。按键的比较结果进行排序，因此迭代器遍历时的顺序是有序的。Query time complexity：O(log n)。由于是平衡二叉搜索树，查找和插入的时间复杂度较稳定`unordered_map` 底层使用使用哈希表（Hash Table）实现，所以不保证元素的任何顺序。迭代器遍历时的顺序可能是无序的。哈希表允许以常数时间复杂度进行平均查找、插入和删除操作。Query time complexity：平均情况下为 O(1)，最坏情况下为 O(n)。哈希冲突可能导致查找时间复杂度的增加。

7. 写法区别

   a. 第一种使用 Operator `[]` 如果 `key` 不在 `imap` 中，`imap[key]` 会默认插入一个具有默认值的键值对，然后将这个值与0比较。这样的写法会在 `imap` 中插入 `key` 对应的值，这可能不是你想要的行为，不会跑出异常。如果你只是想检查键是否存在而不插入新键值对，可能不够安全。

   b. 第二种写法 `find` 函数，`find` 函数会返回一个指向键为 `key` 的元素的迭代器，如果键不存在，则返回 `imap.end()`。这种写法不会修改 `imap`，只是进行查找操作。因此，如果你只是想检查键是否存在，而不想在 `imap` 中插入新键值对，使用 `find` 则更加安全。

编程进阶

1. quick sort 和 merge sort 的区别

Quick Sort 思想是分治，通过选择一个 privot，将数组划分为两个子数组，分别对子数组进行排序，然后合并子数组得到最终排序结果。一般性都是先选取 privot，将数组分为小于privot和大于privot 的两个子数组，递归地对子数组进行排序，相应的不是很稳定。因为在分区过程中，相同元素的相对位置可能会改变。

Avg Time Complexity： O(n log n)

worst case 最差情况发生在每次选择的 privot 都是当前子数组中的最小或最大值，会退化到类似冒泡排序 → O(n^2)

Space Complexity： O(log n)

Merge Sort: 思想是分治也是分治，将数组划分为两个子数组，分别对子数组进行排序，然后将排序好的子数组合并。不断将数组二分，对左右两部分分别排序，最后合并两个有序的子数组。相比较 Quick Sort 来说比较稳定，相同元素的相对位置在排序过程中不会改变。

Avg Time Complexity： O(n log n)

Space Complexity：O(n) 需要额外的空间来存储合并过程中的临时数组。

2. STL 是否线程安全
   C++标准库（STL）并没有要求是线程安全的。
   如果多个线程同时访问或修改STL容器或其他STL组件，没有任何保证会防止数据竞争或其他并发问题，可以使用 Mutex 或者 Parallel computing。
3. 多线程在什么情况下需要加锁，什么情况不需要加锁

   需要加锁的情况：

   1. **并发读写：**
      - 当多个线程同时读取并修改共享数据时，需要使用 `mutex` 来保护共享数据的一致性。
   2. **条件变量的等待和通知：**
      - 当线程需要等待某个条件满足时，通常需要使用条件变量 `condition_variable` 以及 `mutex`。等待和通知过程必须在互斥锁的保护下进行。
   3. **事务操作：**
      - 当多个操作需要作为一个事务来执行时，需要使用互斥锁以确保这个事务的完整性。

   可能不需要加锁的情况：

   1. **只读操作：**
      - 如果所有线程只是读取而不修改共享数据，通常不需要加锁。多个线程可以同时进行读取操作而不会相互影响。
   2. **线程本地数据：**
      - 如果每个线程都拥有自己的私有数据，而不共享给其他线程，那么不需要加锁。

Linux 基础

1. grep chomd chown crontab 作用
    a. `grep`：是一个用于在文本中搜索匹配模式的命令。
    b. `chmod`：它允许用户设置文件的读、写、执行权限。
    c. `chown`：是用于更改文件或目录所有者的命令。它允许用户改变文件或目录的所有者，通常与 `chmod` 结合使用，以便更改文件的权限和所有者。
    d. `crontab`：是用于管理定时任务的命令。它允许用户创建、编辑和删除定期运行的任务。

应用问题

```
#include <iostream>
#include <sstream>
#include <unordered_map>
#include <vector>
#include <mutex>
#include <iomanip>

using namespace std;

struct TickerInfo {
    vector<double> prices;
    double sum = 0.0;
};

unordered_map<string, TickerInfo> tickerData;
mutex dataMutex;

void processTickerData(const char* data) {
    lock_guard<mutex> lock(dataMutex);

    string input(data);
    istringstream iss(input);

    string ticker;
    double price;

    if (getline(iss, ticker, ',') && iss >> price) {
        if (tickerData.find(ticker) == tickerData.end()) {
            tickerData[ticker] = TickerInfo();
        }

        auto& tickerInfo = tickerData[ticker];

        if (tickerInfo.prices.size() >= 20) {
            tickerInfo.sum -= tickerInfo.prices.front();
            tickerInfo.prices.erase(tickerInfo.prices.begin());
        }

        tickerInfo.prices.push_back(price);
        tickerInfo.sum += price;
```

```
        double avgPrice = tickerInfo.sum / tickerInfo.prices.size();
        cout << "Ticker: " << ticker << ", Average Price: " << fixed << setprecision(2) << avgPrice << endl;
    }
}

int main(int argc, const char * argv[]) {
    processTickerData("600000,10.0");
    processTickerData("600000,12.0");
    processTickerData("600000,8.0");

    return 0;
}
```